สคริปต์นำเสนอและเดโมระบบ

ระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้า ร้านวัสดุก่อสร้างอำพรคอนกรีต (SOML)

สำหรับอัดวิดีโอผ่าน Microsoft Teams ส่งอาจารย์ · ความยาวเป้าหมายประมาณ 25–30 นาที

ช่วง	เรื่อง	เวลาโดยประมาณ
0	เตรียมก่อนอัด (ไม่อยู่ในวิดีโอ)	–
1	เปิดตัว	0:30
2	ปัญหาของร้าน	2:30
3	เก็บความต้องการมาอย่างไร	2:00
4	ความต้องการมีอะไรบ้าง แบ่งอย่างไร	3:00
5	จากเรื่องเล่าของผู้ใช้ไปเป็น Use Case	3:00
6	การออกแบบ C1 C2 C3 และฐานข้อมูล	4:00
7	เดโมระบบจริงตามลำดับการขายหนึ่งใบ	9:00
8	การทดสอบและผลลัพธ์	2:30
9	ตารางสืบย้อน: อะไรสัมพันธ์กับอะไร	1:30
10	สิ่งที่ยังไม่เสร็จ และปิดท้าย	1:30

วิธีใช้สคริปต์นี้: อย่าอ่านตามทีละคำ อาจารย์บอกว่าจะดูว่าเราอธิบายเองได้ไหม ให้อ่านช่อง “พูด” จนเข้าใจ แล้วพูดด้วยคำของตัวเอง ช่อง “ทำบนจอ” คือสิ่งที่ต้องคลิกหรือเปิดให้เห็นในวิดีโอ ถ้าพูดผิดหรือติด ไม่ต้องอัดใหม่ทั้งหมด พูดแก้ต่อไปได้เลย เพราะอาจารย์ดูความเข้าใจ ไม่ได้ดูความลื่น

0. เตรียมก่อนอัด (ทำก่อนกดอัด ไม่ต้องอยู่ในวิดีโอ)

#	สิ่งที่ต้องเตรียม	ทำอย่างไร
1	ล้างข้อมูลให้สะอาด	ในโฟลเดอร์ soml รัน npm run db:reset ข้อมูลจะเหลือสินค้า 783 รายการ ผู้ใช้ 4 บัญชี ไม่มีคำสั่งซื้อ
2	เปิดระบบ	รัน npm run dev แล้วรอจนขึ้นว่าหลังบ้านพร้อมที่พอร์ต 4000
3	เบราว์เซอร์สอง หน้าต่าง วางซ้าย-ขวา	หน้าต่างซ้ายปกติ หน้าต่างขวาแบบไม่ระบุตัวตน (Incognito) เพราะสองหน้าต่างต้องเข้าระบบคนละบัญชี
4	เปิดไฟล์ที่ต้องโชว์ไว้ในแท็บ	เอกสารแต่ละบท/บทที่3.pdf และ VS Code เปิดโฟลเดอร์ soml
5	บัญชีที่ใช้	siriphan พนักงานขาย · santi พนักงานคลัง · ning ผู้จัดการ · รหัสผ่านตามค่า SEED_PASSWORD ในไฟล์ .env
6	เครื่องพิมพ์	ถ้าอัดบน Mac ใช้โหมดพิมพ์ลงไฟล์ (PRINTER_INTERFACE=file:///./printer-output) แล้วโชว์ไฟล์ใบเสร็จ .txt ที่ได้ ถ้าอัดบน PC ที่ต่อเครื่องพิมพ์แล้ว โชว์ใบเสร็จจริงได้เลย
7	ฝั่ง Teams	เข้าประชุมคนเดียว › Share screen เลือกทั้งหน้าจอ › More › Start recording › เปิดไมค์ ปิดการแจ้งเตือนของเครื่องก่อน
8	ลองซ้อมหนึ่งรอบ	ซ้อมช่วง 7 ครบหนึ่งรอบ แล้วรัน npm run db:reset อีกครั้งก่อนอัดจริง

1. เปิดตัว (0:30)

ทำบนจอ	พูด
หน้าแรกของ บทที่3.pdf หรือหน้าเข้าสู่ระบบ	สวัสดีครับอาจารย์ ผม [ชื่อ] กับ [ชื่อเพื่อน] โครงงานของเราคือระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้าของร้านวัสดุก่อสร้างอำพรคอนกรีต ชื่อย่อว่า SOML วิดีโอนี้จะเล่าตั้งแต่ปัญหาของร้าน เก็บความต้องการมาอย่างไร ความต้องการมีอะไรบ้าง แบ่งเป็น Use Case อย่างไร ออกแบบ C2 และ C3 อย่างไร แล้วเดโมระบบจริงให้ดูว่าทำงานตามขั้นตอนของ Use Case ครับ

2. ปัญหาของร้าน (2:30)

ทำบนจอ	พูด
บทที่ 3 หน้า 3 ตารางที่ 3.2	ร้านนี้ขายวัสดุก่อสร้าง หน้าร้านกับคลังสินค้าอยู่คนละที่ เดิมทุกอย่างใช้บิลกระดาษ ลูกค้าจ่ายเงินที่หน้าร้าน ได้บิลมาหนึ่งใบ แล้วต้องถือบิลเดินไปคลังเอง
ชี้ EV-01, EV-02	ปัญหาแรกคือการขายไม่ได้เชื่อมกับสต็อก ขายไปแล้วไม่มีอะไรหักยอดในคลัง และบิลกระจัดกระจาย ตรวจย้อนหลังไม่ได้ว่าใบไหนใครขาย
ชี้ EV-03, EV-04	ปัญหาที่สองคือคลังไม่รู้ว่ามีคนซื้อจนลูกค้าเดินมาถึง ช่วงคนเยอะลูกค้าต้องต่อแถวรอที่คลัง
ชี้ EV-05, EV-06, EV-07	ปัญหาที่สามอยู่ที่คลัง พนักงานอ่านลายมือในบิลแล้วหยิบผิด มีคนเอาบิลเก่ามารับของซ้ำ และยอดสต็อกในบันทึกไม่ตรงกับของจริง
ชี้ EV-08, EV-09	ปัญหาสุดท้ายคือผู้จัดการรู้ยอดขายตอนปิดร้านเท่านั้น และสั่งของจากความรู้สึก ของหมดแล้วจึงรู้
ชี้คอลัมน์วัตถุประสงค์	ปัญหาทั้งเก้าข้อ เราให้รหัส EV ย่อจาก Evidence แต่ละข้อผูกกับวัตถุประสงค์ในบทที่ 1 สี่ข้อ คือลดความผิดพลาดจากกระดาษ ลดเวลารอ ทำให้สต็อกตรงและกันรับของซ้ำ และให้ผู้บริหารเห็นข้อมูลทันที ครบทั้งสี่ข้อ ไม่มีวัตถุประสงค์ข้อไหนที่ไม่มีปัญหาจริงรองรับ

3. เก็บความต้องการมาอย่างไร (2:00)

ทำบนจอ	พูด
บทที่ 3 หน้า 2 รูปที่ 3.1	เราทำงานแบบน้ำตกที่วนกลับได้ (Iterative Waterfall) เริ่มจากไปดูการทำงานจริงที่ร้าน ดูว่าบิลเดินจากหน้าร้านไปคลังอย่างไร และถามเจ้าของร้านกับพนักงาน
กลับไปตารางที่ 3.2	สิ่งที่เห็นจริง เราจดเป็นหลักฐานที่มีรหัส ไม่ได้เขียนเป็นความเรียง เหตุผลคือความเรียงเอาไปทดสอบไม่ได้ว่าแก้ปัญหาได้หรือยัง แต่ถ้ามีรหัส จะจับคู่กับความต้องการ Use Case และผลทดสอบได้ทีละข้อ
บทที่ 3 หน้า 3 ย่อหน้าใต้ตาราง 3.2	นอกจากปัญหา เรายังเจอสิ่งที่ต้องรักษาไว้ คือร้านแยกสินค้าเป็นหมวดในไฟล์ Excel และพนักงานคุ้นกับการหาจากหมวดก่อน ระบบจึงมีปุ่มหมวดหมู่บนหน้าขาย แทนที่จะให้พิมพ์ค้นจากสินค้าเจ็ดร้อยกว่ารายการ

4. ความต้องการมีอะไรบ้าง แบ่งอย่างไร (3:00)

ทำบนจอ	พูด
บทที่ 3 หน้า 4 ตารางที่ 3.3	เราแบ่งความต้องการเป็นสามกลุ่ม กลุ่มแรกคือ FR ความต้องการเชิงหน้าที่ คือระบบต้องทำอะไรได้ มีเก้าข้อ ทุกข้อมีคอลัมน์ขวาสุดบอกว่ามาจาก EV ข้อไหน
ชี้ FR-03, FR-04, FR-06	ยกตัวอย่าง FR-03 สร้างคำสั่งซื้อ คำนวณยอด แสดง QR และพิมพ์ใบเสร็จ มาจาก EV-01 FR-04 คิวที่คลังอัปเดตเอง มาจาก EV-03 กับ EV-04 และ FR-06 ตัดสต็อกในธุรกรรมเดียวและกันจ่ายซ้ำ มาจาก EV-06 กับ EV-07
ตารางที่ 3.4 หน้าเดียวกัน	กลุ่มที่สองคือ NFR ระบบต้องทำได้ดีแค่ไหน ทุกข้อต้องมีวิธีวัด เช่น NFR-02 คำสั่งซื้อใหม่ต้องขึ้นที่คลังภายใน 3 วินาที ตัวเลขนี้มาจากที่ลูกค้าใช้เวลาเดินไปคลังราวหนึ่งนาที ไม่กี่วินาทีจึงพอให้คลังเริ่มก่อน ส่วน NFR-06 รองรับ 10 เซสชัน มาจากจำนวนพนักงานที่ทำงานพร้อมกันจริง
บทที่ 3 หน้า 5 ตารางที่ 3.5	กลุ่มที่สามคือ BR กฎทางธุรกิจ คือกติกาของร้านที่เป็นจริงอยู่แล้วแม้ไม่มีคอมพิวเตอร์ เราแยกจาก FR เพราะ BR คือสิ่งที่ระบบต้องเคารพ ส่วน FR คือสิ่งที่ระบบทำเพื่อให้กติกานั้นเป็นจริง เช่น BR-02 สต็อกในระบบต้องตรงของจริง บังคับให้การจ่ายของกับการหักสต็อกต้องอยู่ในธุรกรรมเดียว

5. จากเรื่องเล่าของผู้ใช้ไปเป็น Use Case (3:00)

ทำบนจอ	พูด
บทที่ 3 หน้า 6 ตารางที่ 3.6	ระบบมีผู้ใช้สามบทบาท พนักงานขาย พนักงานคลัง และผู้จัดการ ไม่มีแอดมินแยก เพราะร้านเล็ก งานแอดมินรวมไว้ที่ผู้จัดการ และมีระบบอัตโนมัติอีกหนึ่งตัวที่ทำงานเองโดยไม่มีคนกด ลูกค้าไม่ใช่ผู้ใช้ เพราะไม่มีบัญชีและไม่ได้กดอะไรในระบบ
บทที่ 3 หน้า 7 ตารางที่ 3.7	ก่อนเขียน Use Case เราเขียนเรื่องเล่าของผู้ใช้ หรือ User Story ก่อน ในรูป “ในฐานะ ฉันต้องการ เพื่อ” ให้เจ้าของร้านอ่านแล้วยืนยันได้ว่าตรงใจ เช่น US-03 ในฐานะพนักงานคลัง ฉันต้องการให้ระบบกันจ่ายของซ้ำจากใบเดิม เพื่อจะไม่ต้องจำเองว่าใบไหนจ่ายแล้ว
ตารางที่ 3.8 หน้าเดียวกัน	จากเรื่องเล่า เราแตกเป็น Use Case 11 กรณี คอลัมน์ขวาสุดบอกว่าแต่ละกรณีรองรับ FR ข้อไหน FR ครบทั้งเก้าข้อ ไม่มีข้อไหนที่ไม่มี Use Case รองรับ
ชี้ UC-04	UC-04 สร้างเลขที่คำสั่งซื้อไม่ซ้ำ แยกออกมาจาก UC-03 เพราะมีเงื่อนไขของตัวเองที่ต้องทดสอบแยก คือถ้าพนักงานขายสองคนกดพร้อมกัน เลขต้องไม่ซ้ำ
บทที่ 3 หน้า 8–10 รูปที่ 3.2–3.5	แผนภาพ Use Case แบ่งเป็นสี่รูปตามบทบาท เพราะ 11 กรณีในรูปเดียวอ่านไม่ออก
บทที่ 3 หน้า 11 ตารางที่ 3.10	ทุก Use Case มีขั้นตอนการทำงาน ตัวอย่างนี้คือ UC-07 ยืนยันจ่ายสินค้า ขั้นตอนเหล่านี้คือสิ่งที่โปรแกรมต้องทำตาม เดี๋ยวในช่วงเดโมจะให้ดูว่าโปรแกรมทำตามจริง ขั้นตอนเต็มของทุกกรณีอยู่ในเอกสาร Use Case X หัวข้อ 4
บทที่ 3 หน้า 12 ตารางที่ 3.11	และทุก Use Case มีเกณฑ์การยอมรับ หรือ Acceptance Criteria เขียนเป็นเงื่อนไขตั้งต้น การกระทำ และผลที่ต้องเกิด เช่น AC-07 มีใบรอจ่ายหนึ่งใบ ส่งคำขอจ่ายพร้อมกันสองคำขอ ต้องสำเร็จแค่หนึ่ง และสต็อกหักครั้งเดียว เกณฑ์นี้คือเงื่อนไขที่ระบบต้องผ่านจึงจะยอมรับได้

6. การออกแบบ C1 C2 C3 และฐานข้อมูล (4:00)

ทำบนจอ	พูด
บทที่ 3 หน้า 16 รูปที่ 3.6 (C1)	ภาพกว้างที่สุดคือ C1 บริบทของระบบ มีผู้ใช้สามบทบาท ตัวระบบ SOML และของภายนอกสองอย่าง คือไลบรารีสร้าง QR พร้อมเพย์ กับเครื่องพิมพ์ใบเสร็จความร้อน ระบบไม่ได้เชื่อมกับธนาคาร พนักงานขายดูเงินเข้าจากแอปธนาคารแล้วกดยืนยันเอง
บทที่ 3 หน้า 18 รูปที่ 3.7 (C2) และตารางที่ 3.17	ซูมเข้ามาหนึ่งระดับคือ C2 ระบบแบ่งเป็นสามชั้นเชิงตรรกะ แต่ติดตั้งจริงสี่ส่วน คือหน้าจอบนเบราว์เซอร์ ระบบหลังบ้าน บริการพิมพ์ใบเสร็จ และฐานข้อมูล MySQL
ชี้บริการพิมพ์	เหตุผลที่แยกบริการพิมพ์ออกมาเป็นอีกส่วน ทั้งที่อยู่ชั้นเดียวกับหลังบ้าน คือมันต้องอยู่บนเครื่องที่ต่อสายกับเครื่องพิมพ์เท่านั้น ส่วนหลังบ้านย้ายไปเครื่องไหนก็ได้
ชี้เส้นระหว่างหลังบ้านกับหน้าจอคลัง	อีกเรื่องใน C2 คือช่องทางค้าง หรือ WebSocket หลังบ้านใช้ช่องทางนี้ผลักคำสั่งซื้อใหม่ไปหน้าจอคลังเอง ไม่ต้องรอให้หน้าจอมาถาม ถ้าเปิด WebSocket ไม่ได้ จะเปลี่ยนไปใช้ช่องทางสำรองที่เรียกว่า SSE
บทที่ 3 หน้า 19 รูปที่ 3.8 (C3) และหน้า 13 ตารางที่ 3.13	ซูมเข้าไปในหลังบ้านคือ C3 แบ่งเป็น 10 โมดูลตาม Use Case เช่น Order Controller ดูแล UC-03 กับ UC-04 และ Queue Manager ดูแล UC-05 กับ UC-06
ชี้โมดูล 5 และ 6	ที่ต้องอธิบายคือโมดูล 6 Stock Deduction Service ไม่มีผู้ใช้เรียกตรง ถูกเรียกจาก Dispatch Controller เท่านั้น แยกออกมาเพราะเป็นตรรกะที่อ่อนไหวที่สุดของระบบ ถ้าผิดสต็อกจะเพี้ยน การแยกทำให้เขียนชุดทดสอบเฉพาะได้ และมีความครอบคลุมของการทดสอบร้อยละ 100
VS Code เปิดโฟลเดอร์ soml/backend/modules	ในโค้ดจริง หนึ่งโมดูลคือหนึ่งโฟลเดอร์ ชื่อตรงกับแผนภาพ C3 ทั้ง 10 โฟลเดอร์
บทที่ 3 หน้า 21 รูปที่ 3.9 (ERD)	ฐานข้อมูลมีเจ็ดตาราง มีการตัดสินใจสำคัญสามข้อ ข้อแรก order_items เก็บราคา ณ ตอนขาย เพราะราคาวัสดุเปลี่ยนบ่อย ใบเก่าต้องไม่เปลี่ยนยอดตาม ข้อสอง stock_movements เก็บทุกครั้งที่สต็อกเปลี่ยน เพิ่มได้อย่างเดียว ย้อนดูได้ว่ายอดเพี้ยนตั้งแต่ใบไหน ข้อสาม orders ไม่มีสถานะตะกร้า ใบจะเกิดก็ต่อเมื่อจ่ายเงินแล้วเท่านั้น ยอดขายจึงไม่เพี้ยนจากลูกค้าที่เปลี่ยนใจ
บทที่ 3 หน้า 26 รูปที่ 3.13	คำสั่งซื้อมีสถานะ รอจ่ายสินค้า กำลังจัดของ ส่งมอบสำเร็จ และยกเลิก ส่งมอบสำเร็จเป็นสถานะปลายทาง ย้อนกลับไม่ได้

7. เดโมระบบจริงตามลำดับการขายหนึ่งใบ (9:00)

แนวคิดของช่วงนี้: เดินตามคำสั่งซื้อหนึ่งใบตั้งแต่หน้าร้านจนจ่ายของ ทุกขั้นบอกว่าตรงกับ Use Case ข้อไหน ขั้นตอนไหน และ AC ไหน

7.1 ผู้จัดการตั้งจุดสั่งซื้อเพิ่มไว้ก่อน (UC-01, UC-02)

ทำบนจอ (หน้าต่างซ้าย)	พูด
หน้าเข้าสู่ระบบ › กรอก ning › เข้าสู่ระบบ	เริ่มที่ UC-01 เข้าสู่ระบบในฐานะผู้จัดการ ระบบพาไปหน้าแดชบอร์ด ซึ่งเป็นหน้าแรกของผู้จัดการ
ชี้เมนูด้านซ้าย	ผู้จัดการเห็นเมนูครบเจ็ดเมนู เดี๋ยวพนักงานขายจะเห็นแค่เมนูเดียว
เมนูสินค้า › ค้น 101017 › แก้ไข › จุดสั่งซื้อเพิ่ม ใส่ 1250 › บันทึก	นี่คือ UC-02 จัดการสินค้า เหล็ก 2 หุนเต็มมีอยู่ 1,200 เส้น ผมตั้งจุดสั่งซื้อเพิ่มไว้ที่ 1,250 เพื่อให้เห็นการแจ้งเตือนตอนท้ายเดโม ตามเกณฑ์ AC-02 ค่าที่บันทึกต้องตรงกับที่กรอก และต้องมีบันทึกในประวัติ เดี๋ยวจะเปิดประวัติให้ดูตอนท้าย

7.2 พนักงานคลังเปิดหน้าคิวค้างไว้ (UC-05)

ทำบนจอ (หน้าต่างขวา Incognito)	พูด
เข้าสู่ระบบด้วย santi	หน้าต่างขวาคือพนักงานคลัง ระบบพาไปหน้าคิวรอจ่ายสินค้าทันที เห็นเมนูแค่สามเมนู คือคิว แดชบอร์ด และแจ้งเตือน
ชี้ข้อความ “รับข้อมูลสดจากหน้าร้าน”	ตอนนี้คิวว่าง ข้อความนี้แปลว่าช่องทางค้างเชื่อมต่อแล้ว เปิดหน้านี้ทิ้งไว้

7.3 พนักงานขายขายของ (UC-01, UC-03, UC-04)

ทำบนจอ (หน้าต่างซ้าย)	พูด
ออกจากระบบ › เข้าสู่ระบบด้วย siriphan	ต่อไปผมเป็นพนักงานขาย สังเกตว่าเห็นแค่เมนูขายหน้าร้าน นี่คือการควบคุมสิทธิ์ตามบทบาท และไม่ใช่แค่ซ่อนเมนู ถ้าเรียกเส้นทางของผู้จัดการตรง ๆ เซิร์ฟเวอร์จะปฏิเสธตาม AC-01
กดหมวด เหล็ก › พิมพ์ค้น 101017 › แตะ เหล็ก 2 หุนเต็ม › เปลี่ยนจำนวนเป็น 2	UC-03 ขั้นที่หนึ่ง เลือกหมวด แตะสินค้า ใส่จำนวน (ขายแค่รายการนี้ สินค้าหลายรายการในร้านยังมียอดคงเหลือ 0 ซึ่งแสดงเป็นสีแดง ถ้าใส่ลงไป ตอนจ่ายของระบบจะปฏิเสธเพราะสต็อกไม่พอ)
ชี้ยอดรวม	ขั้นที่สอง ยอดรวมนี้ไม่ได้คิดที่หน้าจอ หน้าจอส่งแค่รหัสสินค้ากับจำนวนไปให้หลังบ้าน หลังบ้านดึงราคาจากฐานข้อมูลมาคิดเอง ใครแก้ราคาที่หน้าจอก็ไม่มีผล
กด “แสดงรหัส QR เพื่อรับชำระ”	ขั้นที่สาม ระบบสร้าง QR พร้อมเพย์ตามยอด ยอดใต้ QR ตรงกับยอดในตะกร้า ตาม AC-03
ชี้ข้อความใต้ปุ่ม	ขั้นที่สี่ถึงห้า ลูกค้าโอนผ่านแอปธนาคารของลูกค้าเอง พนักงานดูเงินเข้าจากแอปธนาคารของร้านแล้วจึงกดยืนยัน ระบบไม่ได้เชื่อมธนาคาร อยู่นอกขอบเขตในบทที่ 1 ปุ่มจึงเขียนว่า “ยืนยันรับชำระเงิน” ไม่ใช่ “ชำระเงิน”
กด “ยืนยันรับชำระเงิน”	ขั้นที่หก ระบบออกเลขที่คำสั่งซื้อ ซึ่งก็คือ UC-04 แล้วบันทึกคำสั่งซื้อ รายการสินค้า และประวัติ ในธุรกรรมเดียว

ทำบนจอ (หน้าต่างซ้าย)	พูด
ชี้เลขที่ ORD-... บนหน้าจอ	เลขรูปแบบ ORD ตามด้วยวันที่และลำดับของวัน ไม่ซ้ำแน่นอน เพราะล็อกเลขล่าสุดก่อนบวกหนึ่ง และถ้าสองเครื่องได้เลขเดียวกัน ฐานข้อมูลจะปฏิเสธแล้วระบบลองใหม่เอง ตาม AC-04
มองหน้าต่างขวา	ขั้นที่เจ็ด ดูหน้าต่างพนักงานคลัง คำสั่งซื้อใหม่ขึ้นเองโดยไม่ได้กดรีเฟรช นี่คือ UC-05 และ AC-06 ที่ต้องขึ้นภายใน 3 วินาที ผลวัดจริงช้าสุด 80 มิลลิวินาที
(Mac) เปิดโฟลเดอร์ soml/printer-output › เปิดไฟล์ .txt ล่าสุด · (PC) โชว์ใบเสร็จจริง	ขั้นที่เจ็ดอีกส่วนคือสั่งพิมพ์ใบเสร็จ ตอนนี้เครื่องนี้ไม่ได้ต่อเครื่องพิมพ์ จึงใช้โหมดพิมพ์ลงไฟล์ นี่คือเนื้อหาที่ส่งไปเครื่องพิมพ์ ถ้าเครื่องพิมพ์ดับ การขายยังบันทึกและเข้าคิวตามปกติ แล้วแจ้งบนจอว่าพิมพ์ไม่สำเร็จ ตาม AC-05 เพราะรับเงินไปแล้ว จะยกเลิกการขายเพราะกระดาษหมดไม่ได้ (ถ้าอัดบน PC ที่ต่อเครื่องพิมพ์แล้ว ให้พูดว่าใบเสร็จออกมาจริงและใช้เวลากี่วินาที)

7.4 พนักงานคลังจัดของและจ่ายของ (UC-06, UC-07)

ทำบนจอ (หน้าต่างขวา)	พูด
กด “เริ่มจัดของ” บนการ์ด	UC-06 ปรับสถานะ ป้ายเปลี่ยนเป็นกำลังจัดของ ทุกจอที่เปิดคิวอยู่เห็นพร้อมกัน คนอื่นจะได้ไม่หยิบซ้ำ ระบบเช็กว่าเปลี่ยนสถานะตามลูกศรในแผนภาพสถานะเท่านั้น
กด “จ่ายสินค้า ›”	UC-07 ขั้นที่หนึ่งถึงสอง หน้านี้แสดงจำนวนที่ต้องจัดตัวใหญ่ และยอดในคลังข้าง ๆ ให้เทียบว่าของพอไหม พนักงานไม่ต้องอ่านลายมืออีกแล้ว ซึ่งแก้ EV-05
กด “ยืนยันจ่ายสินค้า”	ขั้นที่สาม ระบบถามยืนยันอีกครั้ง เพราะจ่ายแล้วย้อนกลับไม่ได้
ก่อนกดยืนยัน: หน้าต่างซ้ายเข้าสู่ระบบเป็น ning › คิว › เปิดใบเดียวกัน › กด “ยืนยันจ่ายสินค้า” ให้ทั้งสองหน้าต่างค้างอยู่ที่กล่องยืนยัน	ตรงนี้จะโชว์ปัญหาจ่ายซ้ำ ผมให้ผู้จัดการเปิดใบเดียวกันอีกหน้าต่าง เหมือนคนสองคนจะจ่ายใบเดียวกัน
กด “ยืนยัน จ่ายสินค้าเลย” หน้าต่างขวา แล้วตามด้วยหน้าต่างซ้าย	ขั้นที่สี่ถึงหก หน้าต่างขวาสำเร็จ ขึ้นว่าส่งมอบสำเร็จ ตัดสต็อกแล้ว หน้าต่างซ้ายถูกปฏิเสธ บอกว่าคำสั่งซื้อถูกจ่ายไปแล้ว นี่คือ AC-07
VS Code เปิด backend/modules/stock-deduction/index.js บรรทัด 32 และ 36	เหตุผลที่กันได้อยู่ที่บรรทัด 32 คำสั่งเปลี่ยนสถานะมีเงื่อนไขว่าสถานะเดิมต้องยังเป็นรอจ่ายหรือกำลังจัด คนแรกเปลี่ยนได้ คนที่สองหาแถวที่ตรงเงื่อนไขไม่เจอ บรรทัด 36 จึงยกเลิกทั้งธุรกรรม สต็อกจึงหักครั้งเดียว เทคนิคนี้คือการล็อกเชิงบวก (Optimistic Locking) ในบทที่ 2 หัวข้อ 2.1.3

ทำบนจอ (หน้าต่างขวา)	พูด
ชี้บรรทัด 42 และ 51	บรรทัด 51 หักสต็อกเฉพาะเมื่อของพอ ถ้าไม่พอ ธุรกรรมทั้งหมดถูกยกเลิก สถานะกับสต็อกไม่เปลี่ยน ตาม AC-08 ส่วนบรรทัด 42 ล็อกสินค้าเรียงตามรหัส เพื่อไม่ให้สองธุรกรรมรอกันเองไม่จบ
(ไม่บังคับ) หน้าต่างซ้ายเป็น siriphan ขายสินค้าที่คงเหลือ 0 หนึ่งรายการ › หน้าต่างขวาเปิดใบนั้น	ถ้ามีเวลา โชว์ AC-08 ได้ เปิดใบที่สั่งของที่ไม่มีในคลัง ระบบขึ้นว่าสต็อกไม่พอ และไม่มีปุ่มยืนยันจ่ายให้กด

7.5 ผู้จัดการเห็นผลทันที (UC-08, UC-11, UC-10, UC-09)

ทำบนจอ (หน้าต่างซ้าย ning)	พูด
ชี้ตัวเลขบนเมนูแจ้งเตือน	จ่ายเหล็ก 2 หุนไปแล้ว ยอดเหลือต่ำกว่าจุดสั่งซื้อเพิ่ม 1,250 ที่ตั้งไว้ตอนต้น ระบบตรวจเองหลังสต็อกเปลี่ยน แล้วสร้างการแจ้งเตือน ตัวเลขบนเมนูขึ้นทันที นี่คือ UC-11 และ AC-10 ตามกฎ BR-05 ว่าต้องเตือนเชิงรุก
เมนูแดชบอร์ด	UC-08 แดชบอร์ดปฏิบัติการ บนสุดคือกล่องต้องดูแลวันนี้ รวมเรื่องที่ต้องจัดการ ตอนนี้มีสินค้าต่ำกว่าจุดสั่งซื้อเพิ่ม และการแจ้งเตือนที่ยังไม่อ่าน ถ้ามีใบรอเกิน 30 นาที ก็จะขึ้นในกล่องนี้ด้วย กล่องนี้ไม่ได้เพิ่มความต้องการใหม่ เป็นข้อมูลของ FR-07 กับ FR-08 ที่มีอยู่แล้ว เอามาไว้ในที่เดียว
ชี้ยอดขายวันนี้และ “คำนวณสดเมื่อ”	ทุกตัวเลขคำนวณสดจากฐานข้อมูลทุกครั้งที่เปิด ไม่เก็บค่าสรุปไว้ ตามกฎ BR-04 ทดสอบแล้วว่าตรงกับผลรวมที่คำนวณจากฐานข้อมูลโดยตรง ตาม AC-09
เมนูแจ้งเตือน › กด “อ่านแล้ว”	ผู้จัดการกดอ่านแล้วได้ ตัวเลขบนเมนูลดลง พนักงานคลังอ่านได้อย่างเดียว กดไม่ได้
เมนูประวัติ › ประเภทการกระทำ “ทั้งหมด”	UC-10 ประวัติการทำรายการ เห็นครบทุกอย่างที่เพิ่งทำ แก้ไขสินค้า สร้างคำสั่งซื้อ เปลี่ยนสถานะ ยืนยันจ่าย และระบบสร้างการแจ้งเตือน พร้อมผู้กระทำ เวลา และหมายเลขไอพี แสดงเป็นชื่อสินค้าหรือเลขที่ใบที่อ่านออก ไม่ใช่รหัสตัวเลข ตาม AC-11 และแก้ปัญหา EV-02
เมนูระดับบริหาร › กด 7 วัน / 30 วัน	UC-09 แดชบอร์ดระดับบริหาร กราฟยอดขายรายวันและสินค้าขายดีสิบอันดับ ใช้ตัดสินใจสั่งของจากข้อมูลจริง แก้ EV-09

7.6 ใช้บนมือถือ และโหมดมืด (NFR-03)

ทำบนจอ	พูด
หน้าต่างขวา กด F12 › Ctrl+Shift+M เลือกมือถือ › หน้าคิว	พนักงานคลังเดินอยู่ในโกดัง ใช้มือถือหรือแท็บเล็ต ตาม NFR-03 พอจอแคบ เมนูย้ายขึ้นด้านบน การ์ดเรียงเป็นคอลัมน์เดียว ปุ่มใหญ่พอกดด้วยนิ้ว
กดปุ่มโหมดมืด	ทุกหน้าจอสลับโหมดสว่างกับโหมดมืดได้ เพราะหน้าร้านสว่างแต่โกดังแสงน้อย เปลี่ยนแค่สี ไม่กระทบข้อมูล

8. การทดสอบและผลลัพธ์ (2:30)

ทำบนจอ	พูด
บทที่ 3 หน้า 33 ตารางที่ 3.23	การทดสอบมีสามระดับ ระดับหน่วยทดสอบตรรกะในโมดูลโดยไม่แตะฐานข้อมูล ระดับการรวมส่วนทดสอบโมดูลกับฐานข้อมูลจริง และระดับระบบทดสอบผ่านหน้าจอจริงเหมือนผู้ใช้กด กรณีทดสอบ TC ทุกข้อผูกกับ AC
terminal ในโฟลเดอร์ soml รัน npm test	สองระดับแรกมี 134 ข้อ รันอัตโนมัติ ผ่านทั้งหมด ใช้เวลาราวสี่วินาที ความครอบคลุมของโค้ดราวร้อยละ 90 โมดูลตัดสต็อกครอบคลุมร้อยละ 100
บทที่ 4 หน้า 17 ตารางที่ 4.4	ระดับระบบมี 13 ข้อ ใช้ Playwright เปิดเบราว์เซอร์กดจริง ครอบคลุม AC-01 ถึง AC-11 ผ่านทั้งหมด
บทที่ 4 หน้า 18 ตารางที่ 4.5	ผลตาม TC-01 ถึง TC-13 เช่น TC-04 สร้าง 120 ใบทั้งต่อเนื่องและพร้อมกัน เลขไม่ซ้ำ TC-05 ขึ้นที่คลังช้าสุด 80 มิลลิวินาที TC-13 สิบเซสชันพร้อมกัน 15,383 คำขอ ไม่ล้มเหลวเลย

9. ตารางสืบย้อน: อะไรสัมพันธ์กับอะไร (1:30)

ทำบนจอ	พูด
บทที่ 3 หน้า 12 ตารางที่ 3.12 หรือ บทที่ 4 ตารางที่ 4.8	สุดท้ายคือทุกอย่างต่อกันอย่างไร ยกแถว EV-06 ร้านเคยโดนเอาบิลเก่ามารับของซ้ำ ทำให้เกิด FR-06 ตัดสต็อกในธุรกรรมเดียวและกันจ่ายซ้ำ ภายใต้กฎ BR-02 พนักงานคลังพูดไว้ใน US-03 ระบบทำใน UC-07 ขั้นที่สี่ ยอมรับเมื่อผ่าน AC-07 ทดสอบด้วย TC-06 ผ่าน ผู้ใช้ทำที่หน้ายืนยันจ่ายสินค้า ข้อมูลอยู่ในตาราง orders products และ stock_movements และโค้ดอยู่ในโมดูล Stock Deduction ที่เพิ่งเปิดให้ดู
	แต่ละ Use Case จึงมีครบห้าอย่าง คือ ความต้องการที่รองรับ ขั้นตอนที่โปรแกรมทำตาม เกณฑ์การยอมรับ หน้าจอ และข้อมูลที่ใช้ ซึ่งรวมกันเป็น ERD

10. สิ่งที่ยังไม่เสร็จ และปิดท้าย (1:30)

ทำบนจอ	พูด
บทที่ 4 ตารางที่ 4.5	กรณีทดสอบทั้ง 13 ข้อผ่านหมด ข้อที่ต้องใช้อุปกรณ์จริงคือ TC-12 พิมพ์ใบเสร็จกับเครื่องพิมพ์จริงผ่าน USB ภาษาไทยถูก ใช้เวลาประมาณ 2.3 วินาที
	อีกเรื่องที่รู้แล้วคือ UC-06 กับ UC-09 ยังไม่มีเกณฑ์การยอมรับของตัวเอง มีแค่การทดสอบระดับการรวมส่วน และยังไม่ได้ให้คนนอกลองติดตั้งตามคู่มือ กับยังไม่ได้ทำแบบประเมินความพึงพอใจจากผู้ใช้จริง
หน้าแดชบอร์ด	สรุปคือระบบแก้ปัญหาทั้งเก้าข้อที่เจอที่ร้าน ความต้องการทุกข้อสืบกลับไปหาปัญหาได้ และทุกข้อมีผลทดสอบ ขอบคุณครับอาจารย์

ภาคผนวก: ถ้าเดโมติดขัด

อาการ	แก้
การ์ดไม่ขึ้นที่หน้าคิวเอง	ดูว่าข้อความเป็น “รับข้อมูลสดจากหน้าร้าน” หรือยัง ถ้ายัง กดรีเฟรชหน้าคิวหนึ่งครั้งแล้วขายใหม่
เข้าระบบสองบัญชีแล้วบัญชีทับกัน	หน้าต่างที่สองต้องเป็นแบบไม่ระบุตัวตน (Incognito)
ตัวเลขแจ้งเตือนไม่ขึ้น	ตรวจว่าตั้งจุดสั่งซื้อเพิ่มสูงกว่ายอดคงเหลือแล้ว การแจ้งเตือนจะเกิดตอนจ่ายของ ไม่ใช่ตอนแก้จุดสั่งซื้อเพิ่ม
ข้อมูลรกจากการซ้อม	หยุดอัด รัน npm run db:reset แล้วเริ่มช่วง 7 ใหม่
หลังบ้านขึ้น EADDRINUSE	มีตัวเก่าเปิดค้าง ปิดหน้าต่าง terminal เก่าแล้วรัน npm run dev ใหม่